

## *Expt. No. 1*

**Title:** *Formulate Mini Project Problem Statement.*

| <i>Rubric</i>                                             | <i>Score (0 to 5)</i> |
|-----------------------------------------------------------|-----------------------|
| <i>In time Submission</i>                                 |                       |
| <i>Clean, Commented and Correctness of code</i>           |                       |
| <i>Concept Understanding<br/>(ability to answer viva)</i> |                       |
| <b><i>Total</i></b>                                       |                       |

***Performed On:***

***Sign:*** \_\_\_\_\_

# Experiment No. 1

**Aim:** Identify a real-world NLP problem, such as sentiment analysis or chatbot development, and frame a detailed problem statement for a mini project.

## Pre-Lab Questions:

1. What is a real-world problem that can be addressed using NLP techniques? Provide some examples.  
NLP can automate text understanding tasks such as spam email detection, sentiment analysis of reviews, chatbots for customer support, language translation, and text summarization.
2. Why is it important to clearly define the problem statement before starting an NLP project?  
A clear problem statement helps identify the right data, NLP techniques, and evaluation metrics, ensuring the solution meets the intended objective.
3. Choose a potential NLP application (e.g., sentiment analysis, chatbot, text summarization). What are the expected benefits of implementing this application?  
Sentiment Analysis: It helps businesses understand customer opinions, improve products and services, and make data-driven decisions based on user feedback.

## Post-Lab Questions:

1. What are the key challenges that may arise when developing your chosen NLP solution (e.g., data availability, model accuracy, understanding context)?
2. How would you determine the scope of your mini project to ensure it's achievable within a limited timeframe?

**Software:** 1. Python, 2. Jupyter Notebook 3. VS Code, 4. NLTK 5. spaCy, 6. TextBlob, 7. Genism, 8.

Hugging Face Transformers, 9. scikit-learn 10. TensorFlow, 11. PyTorch, 12. Pandas, 13.

NumPy, 14. Matplotlib, 15. Seaborn 16. WordCloud, 17. Google Colab, 18. AWS, Azure, GCP, 19.

Google Translate API 20. Microsoft Azure Translator, 21. OpenAI GPT API, 22. Git,

23. GitHub

## Theory:

A well-defined problem statement for an NLP project must include the following details:

1. **Problem Description:** Clearly describe the problem you aim to solve, providing context and background. Explain why the problem exists and its significance.
2. **Objectives:** Define the specific goals of the project. What outcome do you aim to achieve? For example, improving sentiment classification accuracy or automating a particular language-based task.
3. **Scope:** Outline the boundaries of the problem. Specify what is included and what is excluded, ensuring the project remains feasible and focused.
4. **Target Audience:** Identify the intended users or beneficiaries of the solution. Who will benefit from the implementation of this project?
5. **Data Requirements:** Specify the type of data needed for the project (e.g., text reviews, chat logs).

Mention the data sources and any preprocessing requirements.

6. **Challenges and Assumptions:** Highlight the key challenges, such as data availability, ambiguity, and language-specific issues. Mention any assumptions that guide your approach to solving the problem.
7. **Evaluation Metrics:** Define the criteria that will be used to evaluate the success of the project (e.g., accuracy, precision, recall, BLEU score).
8. **Tools and Techniques:** Mention the specific NLP techniques, machine learning algorithms, and software tools that will be used to develop the solution.
9. **Impact and Significance:** Describe the potential impact of the project. How will solving this problem benefit users or address a gap in current technology?
10. **Deliverables:** Define the expected output or deliverables from the project, such as a trained model, application prototype, or detailed report.

### **Problem statement:**

## **RecallAI - Project Documentation**

### **1. Problem Description**

Students often struggle to revise lengthy lecture notes because manually identifying important concepts, preparing summaries and creating practice questions requires considerable time and effort. Traditional note-taking methods provide limited support for automated revision and self-assessment. This project proposes RecallAI, an NLP-based application that analyzes educational text, extracts important concepts, generates concise summaries and automatically creates flashcards and practice questions. The proposed system aims to simplify revision preparation, encourage active recall and provide students with an interactive learning experience.

### **2. Objectives — RecallAI**

1. **Automate Content Summarization:** Develop an NLP-based system that transforms lengthy educational notes into concise and meaningful summaries.
2. **Extract Key Information:** Identify important concepts, keywords and definitions from uploaded documents.
3. **Generate Intelligent Questions:** Automatically create relevant MCQs, short-answer questions and flashcards from the source content.
4. **Ensure Content Accuracy:** Verify generated answers against the original notes to minimize hallucinations and misinformation.
5. **Enhance Knowledge Retention:** Support active recall and self-assessment through interactive quizzes and flashcards.
6. **Develop a User-Friendly Application:** Build an accessible web interface for uploading notes and generating structured revision materials.

### 3. Scope

#### A. In Scope

1. Educational Notes Input: Allow students to upload TXT files or paste English lecture notes directly into the application.
2. Automatic Text Summarization: Convert lengthy lecture notes into concise, meaningful summaries while preserving important concepts.
3. Keyword and Concept Extraction: Identify important terms, definitions and key concepts from educational content using NLP techniques.
4. Intelligent Flashcard Generation: Automatically generate question-and-answer flashcards to help students revise and memorize important information.
5. Automated Quiz Generation: Generate multiple-choice questions and short-answer questions based on uploaded notes, along with correct answers.

#### B. Out of Scope

- 1. Handwritten Notes Recognition:** The system will not recognize handwritten notes or extract text from scanned documents.
- 2. Audio and Video Processing:** The application will not convert recorded lectures, audio files or educational videos into text.
- 3. Multilingual Support:** The initial version will support English educational content only, excluding Hindi, Marathi and other languages.
- 4. Training Large Language Models from Scratch:** The project will use existing pretrained NLP models instead of developing and training a completely new language model.
- 5. Advanced Learning Management System:** Features such as student attendance, course enrollment, classroom management and teacher administration will not be included.
- 6. Automatic Essay Evaluation:** The system will not evaluate lengthy descriptive answers or provide detailed grading for essays.

#### 4. Target Audience

- 1. Students and Competitive Exam Aspirants:** Students who need to convert lengthy study materials into concise summaries, flashcards and practice questions for efficient revision.
- 2. Teachers and Educators:** Educators who want to automatically generate quizzes, important questions and revision materials from lecture notes.
- 3. Business Professionals and Executives:** Professionals who need to summarize lengthy meeting notes, identify important discussions and extract key decisions.
- 4. Entrepreneurs and Business Owners:** Business owners who want to organize meeting information, highlight important insights and prepare actionable follow-ups.
- 5. Corporate Employees and Trainers:** Employees and trainers who need to transform training materials, workshops and meeting notes into summaries, quizzes and knowledge-retention exercises.

## 5. Data Requirements — RecallAI

### A. Data Types Needed

1. **Educational Text Data:** Lecture notes, textbook content, study materials and educational articles containing definitions, concepts and explanations.
2. **Meeting and Training Notes:** Text-based meeting minutes, business discussions and corporate training materials for future summarization and knowledge extraction.
3. **Question–Answer Datasets:** Educational questions and their corresponding answers to evaluate automatically generated quizzes and flashcards.

### B. Data Sources

1. **User-Uploaded Notes:** English text entered or uploaded by students, teachers and professionals.
2. **Public Educational Datasets:** Open-source educational datasets from platforms such as Kaggle and Hugging Face.
3. **Sample Meeting Documents:** Manually prepared or appropriately authorized meeting notes and training materials.
4. **Manually Curated Dataset:** A collection of sample notes, reference summaries and question–answer pairs for testing and evaluation.

### C. Data Preprocessing Requirements

- Text Cleaning: Remove unnecessary symbols, extra spaces, formatting errors and irrelevant characters.
- Tokenization: Divide text into sentences and words for further NLP processing.
- Text Segmentation: Split lengthy documents into smaller, meaningful sections.
- Keyword Extraction: Identify important terms, definitions and concepts.
- Data Validation: Check whether the input contains sufficient readable text and remove duplicate or irrelevant content.

## 6. Challenges and Assumptions

### A. Key Challenges

1. **Context Understanding:** Identifying important concepts while preserving their original meaning during summarization.
2. **Question Quality:** Generating meaningful, non-repetitive questions with accurate answers.
3. **Hallucination Risk:** Preventing AI from generating information that is not present in the uploaded notes.
4. **Document Complexity:** Handling lengthy, poorly structured or incomplete study materials.
5. **Model Performance:** Maintaining acceptable accuracy and processing speed with limited computational resources.

### B. Assumptions

1. Uploaded documents contain readable, English-language text.
2. Input notes provide sufficient information for generating summaries and questions.
3. Pretrained NLP models are suitable for developing the initial prototype.
4. Generated content will be based on the uploaded material and evaluated for factual correctness.

## 7. Evaluation Metrics

1. **ROUGE Score:** Measures how well generated summaries match reference summaries.
2. **BERTScore:** Evaluates semantic similarity between generated and reference summaries.
3. **Question Relevance:** Measures whether generated questions are meaningful and related to the uploaded content.
4. **Answer Accuracy:** Evaluates the correctness of generated answers against the original notes.
5. **Content Faithfulness:** Checks whether summaries and questions contain unsupported or hallucinated information.
6. **Processing Time:** Measures the time required to generate summaries, flashcards and quizzes.

## 8. Tools and Techniques

### A. NLP Techniques

1. **Text Preprocessing:** Tokenization, text cleaning and sentence segmentation.
2. **Text Summarization:** Generate concise summaries using pretrained transformer models.
3. **Keyword Extraction:** Identify important concepts and terms using KeyBERT.
4. **Question Generation:** Automatically generate MCQs, short-answer questions and flashcards.
5. **Answer Verification:** Check generated answers against the original notes to reduce hallucinations.

### B. Software and Frameworks

1. **Python:** Core programming language.
2. **NLTK and spaCy:** Text preprocessing and linguistic analysis.
3. **Hugging Face Transformers:** Pretrained models for summarization and question generation.
4. **Pandas:** Data processing and evaluation.
5. **Git and GitHub:** Version control and project documentation.

## 9. Impact and Significance

1. **Time Efficiency:** Reduces manual effort in preparing summaries, flashcards and practice questions.
2. **Improved Learning:** Encourages active recall and self-assessment through AI-generated quizzes.
3. **Knowledge Accessibility:** Converts lengthy and complex notes into concise, structured learning materials.
4. **Personalized Revision:** Enables users to generate revision content based on their own uploaded materials.
5. **Business Applications:** Provides a foundation for future meeting summarization, key decision extraction and corporate training.
6. **Practical NLP Application:** Demonstrates how summarization, keyword extraction and question generation can solve real-world problems.

## 10. Deliverables

- 1. Functional Web Application:** A user-friendly RecallAI application for uploading and processing notes.
- 2. Text Summarization Module:** Automatically generates concise summaries from lengthy documents.
- 3. Keyword Extraction Module:** Identifies important concepts, definitions and key information.
- 4. Flashcard and Quiz Generator:** Creates flashcards, MCQs and short-answer questions with answers.
- 5. Answer Verification System:** Checks generated answers against the original content to reduce inaccuracies.
- 6. Evaluation Report:** Documents the system's performance, accuracy, relevance and processing time.
- 7. Project Documentation:** Complete source code, GitHub repository, technical report and final presentation.

## Conclusion

RecallAI addresses the challenges of manual note summarization and revision preparation by proposing an NLP-based system that transforms lengthy educational content into concise summaries, flashcards and practice questions. The project combines text summarization, keyword extraction and question generation to support efficient learning and active recall. Its future scope includes business meeting summarization and corporate training applications. This experiment successfully established a well-defined problem statement, objectives, scope, methodology and evaluation criteria, providing a foundation for the development of RecallAI.

## CO's Covered

CO5: Apply NLP techniques to design real-world applications such as text summarization, information extraction and question-answering systems.

and return the correct result when compared against the curated benchmark set

1. **Query Routing Accuracy:** accuracy/confusion matrix of the classifier that decides SQL vs. semantic vs. hybrid retrieval path
2. **Answer Faithfulness** (RAGAS-style): whether the generated natural language answer accurately reflects the retrieved data/SQL results, without hallucinated additions
3. **Answer Relevance** (RAGAS-style): whether the response actually addresses the user's question
4. **Retrieval Precision/Recall:** for the semantic search component, whether the correct rows/columns are retrieved for a given free-text query
5. **Outlier Detection Effectiveness:** qualitative check on whether flagged outliers are genuinely anomalous (validated against known distributions in test datasets)
6. **Latency:** average response time per query, particularly relevant given local hardware constraints

## 2. Tools and

### Techniques NLP/ML

#### Techniques:

- Text-to-SQL generation via prompt-engineered LLM calls
- Sentence embeddings for semantic search (sentence-transformers, all-MiniLM-L6-v2)
- Query classification (rule-based initially, potentially a lightweight trained classifier)
- Statistical outlier detection (IQR, z-score, Isolation Forest)
- Automated exploratory data profiling (missing values, distributions, correlations)

#### Software/Frameworks:

- **Backend:** Flask
- **Database:** PostgreSQL with pgvector extension (unified structured + vector storage)
- **LLM:** Ollama (local, Llama 3.2 3B / Phi-3 Mini) or OpenAI GPT-4o-mini / Groq (free tier), swappable via configuration
- **SQL Execution:** SQLAlchemy (safe, parameterized execution)
- **Data Science Libraries:** pandas, scikit-learn
- **Frontend:** Flask + Jinja templates (or lightweight React panel)
- **Version Control:** Git & GitHub

## 3. Impact and Significance

DM Your Data addresses a genuine gap between two mature but separately evolved technologies: traditional relational databases (precise but requiring technical expertise) and modern conversational AI (accessible but often unreliable on structured data). By grounding every answer in verifiable SQL execution rather than LLM approximation, the project offers a trustworthy alternative to naive "upload your CSV to ChatGPT" workflows, directly addressing accuracy, privacy, and scalability concerns that limit current casual

approaches. It also fills a gap in the existing open-source ecosystem, where most community text-to-SQL

projects handle only structured querying and ignore free-text data entirely, and where formal evaluation of accuracy is rarely built in. For its target users, this translates into faster, self-serve access to insights without needing SQL skills or a dedicated analyst, while for the AI/data science community, it demonstrates a reproducible, evaluable architecture for hybrid RAG over tabular data.

#### **4. Deliverables**

1. A fully functional Flask web application with PostgreSQL backend for natural language querying of tabular data
2. Automated data profiling module generating EDA summaries on data upload
3. Text-to-SQL and semantic retrieval pipeline, including the query classification/routing logic
4. Outlier detection layer integrated into aggregate query responses
5. Evaluation harness and report: benchmark dataset, accuracy metrics, and analysis of failure modes
6. Evaluation dashboard visualizing RAG/SQL accuracy metrics over time
7. Technical documentation/README covering architecture, setup instructions, and design decisions
8. Final project report and presentation summarizing methodology, results, and future scope

#### **Conclusion:**

DM Your Data bridges the gap between precise but technical SQL tools and accessible but unreliable LLM-based approaches like uploading a CSV to a chatbot, by combining text-to-SQL retrieval with semantic vector search under one intelligent query router, so every answer stays grounded in an actual, auditable SQL query rather than LLM guesswork. What sets it apart is not the core idea, which is already established, but its completeness: automated profiling, outlier detection, schema linking, and a formal evaluation harness to measure accuracy, an evaluation-first approach most similar open-source projects skip. Scoped around a single-user, batch-upload workflow, the project proves this hybrid architecture works and can be measured reliably, resulting in a functional, self-hostable application that shows how classical data science and generative AI can combine to build systems that are genuinely trustworthy, not just impressive in a demo.

#### **CO's Covered:**

CO 5) To apply NLP techniques to design real world NLP applications such as machine translation, sentiment analysis, text summarization, information extraction, Question Answering system etc.